

Atividade Prática

Aula 3 — Introdução a Padrões de Projeto e Princípios SOLID

Técnicas de Programação II (TP2) · CST em Desenvolvimento de Software Multiplataforma · Fatec de Porto Ferreira · Prof. Vagner dos Santos

1. Identificação da atividade

Campo	Preenchimento
Aula	Aula 3 — Padrões de Projeto e SOLID
Título da atividade	Refatoração do SIGA aplicando princípios SOLID
Data de aplicação	26/08/2026
Prazo de entrega	02/09/2026, 19h10
Modalidade	Individual
Duração em aula	30 minutos
Peso na avaliação	Compõe a nota de atividades práticas conforme o plano de avaliação da disciplina

2. Objetivo da atividade

Ao final desta atividade, o estudante deverá ser capaz de identificar violações dos princípios SOLID e refatorar o código para respeitá-los, demonstrando na prática a relação entre SOLID, coesão e acoplamento.

3. Contexto

Esta atividade evolui o Sistema de Gestão Acadêmica Simplificado (SIGA) e parte da situação-problema da aula: a classe `Matricula` calcula a mensalidade com um bloco de condicionais que cresce a cada novo tipo de desconto, exigindo modificação e reteste a cada semestre.

4. Enunciado e etapas

Enunciado: A partir do código do SIGA, identifique violações dos princípios SOLID e refatore-o para respeitá-los, tornando-o aberto para extensão e de baixo acoplamento.

Execute as etapas na ordem, registrando a evidência esperada de cada uma:

Etapas	Ação do estudante	Evidência esperada
1	Analisar a classe <code>RelatorioAluno</code> , que acumula persistência, formatação e envio.	Identificação, por escrito, das responsabilidades misturadas (violação do SRP).
2	Refatorar separando cada responsabilidade em sua própria classe.	Responsabilidade única aplicada (SRP).
3	Substituir o bloco condicional por tipo de desconto por polimorfismo.	Solução aberta para extensão sem modificação (OCP), via interface <code>Desconto</code> .
4	Introduzir uma interface e inverter uma dependência concreta.	Dependência de abstração, não de implementação (DIP).
5	Listar os code smells encontrados no código original.	Diagnóstico documentado dos indícios de mau cheiro de código.

5. Regras de produção (fixas)

Estas regras valem para todas as atividades práticas da disciplina.

5.1 Padrão de código

- Linguagem: Java (JDK 17 ou superior).
- Identificadores (classes, atributos, métodos, variáveis) em português, seguindo o domínio do problema (ex.: `Aluno`, `matricular`, `calcularMedia`). Nomes claros e significativos, sem abreviações obscuras.
- Convenções Java: classes em PascalCase (`Aluno`, `MatriculaService`); métodos e atributos em camelCase (`calcularMedia`, `nomeCompleto`); constantes em UPPER_SNAKE_CASE.
- Indentação consistente (4 espaços), chaves e espaçamento padronizados. O código deve estar formatado pela IDE antes da entrega.
- Um arquivo `.java` por classe pública, com o nome do arquivo igual ao da classe.

5.2 Organização do projeto

- O código deve compilar e executar sem erros. Código que não compila não será avaliado nas etapas dependentes.
- Separação de responsabilidades conforme trabalhado em aula (entidades, acesso a dados, serviços, apresentação), evitando classes que acumulam múltiplas responsabilidades.
- Sem código morto, comentado em bloco ou trechos de teste esquecidos na entrega final.

5.3 Uso de ferramentas de IA

- O uso de assistentes de IA (Copilot, ChatGPT e similares) para gerar o código da atividade é PROIBIDO por padrão, pois o objetivo é o desenvolvimento da competência individual.
- A proibição só é dispensada quando o enunciado da aula indicar explicitamente o uso de IA; nesse caso, o aluno deve declarar o que foi gerado com auxílio de IA e como foi validado.
- Consultar documentação oficial da linguagem e materiais da disciplina é sempre permitido.

6. Requisitos de entrega (fixos)

- Meio de entrega: repositório Git da disciplina (padrão). O registro no SIGA/Teams, quando exigido, aponta para o repositório.
- Estrutura mínima do repositório: pasta da atividade identificada pela aula, com o código-fonte e um arquivo README.md descrevendo como compilar e executar.
- Commits com mensagens claras e em português, refletindo o progresso; evitar um único commit final com todo o trabalho.
- Entregar o código E as evidências pedidas na Seção 4 (prints, saída de execução ou diagrama, conforme a etapa).
- Em atividades em dupla, ambos os integrantes devem ter commits no repositório; a ausência de commits de um integrante impacta sua nota individual.

7. Critérios de avaliação — rubrica (fixa)

Cada critério é avaliado em três níveis. A nota da atividade resulta da soma ponderada dos critérios.

Critério	Insuficiente	Adequado	Excelente
Funcionamento e compilação	Não compila ou não executa	Compila e executa com pequenas falhas	Compila, executa e atende a todos os casos
Aplicação do conceito da aula	Conceito ausente ou incorreto	Conceito aplicado com imprecisões	Conceito aplicado corretamente e com clareza
Qualidade e organização do código	Desorganizado, sem padrão	Organizado, com padrão razoável	Coeso, bem nomeado e formatado
Cumprimento das etapas e evidências	Etapas/evidências ausentes	Maioria das etapas com evidência	Todas as etapas com evidência completa
Entrega (Git, README, commits)	Fora do padrão de entrega	Entrega no padrão com lacunas	Entrega completa e bem documentada

Pontuação: cada critério vale de 0 a 2 pontos (Insuficiente = 0; Adequado = 1; Excelente = 2), totalizando 10 pontos.

8. Integridade acadêmica e política de atraso (fixas)

- O trabalho deve ser de autoria do(s) estudante(s). Cópia entre colegas, no todo ou em parte, caracteriza plágio e zera a atividade dos envolvidos.
- Compartilhar a própria solução para cópia também constitui violação de integridade.
- O uso não declarado de IA, quando esta não foi autorizada pelo enunciado, é tratado como violação de integridade.
- Entregas após o prazo: penalidade de -20% por dia de atraso, até o limite de 2 dias; após isso, a atividade não é aceita.
- Dúvidas sobre o enunciado devem ser esclarecidas com o professor antes do prazo, preferencialmente durante a aula.